

Infografía

Módulo 10 — Shaders · Sesión 3

La pantalla entera: post-procesamiento, pausa en gris y luz 2D

UPB · Gestión I/2026

Plan de la clase

1. **El shader de pantalla** — leer el frame y repintarlo (escena 12).
2. **Viñeta y CRT** — con las teclas 0/1/2.
3. **Pausa en gris** — módulo 9 + shader (escena 13, en vivo).
4. **Luz 2D + normal maps** — volumen sin 3D (escena 14).
5. **Ejercicio síntesis** — la viñeta de daño (escena 15).
6. **Cierre del módulo** — qué te llevas al proyecto final.

Recap de 20 segundos: un uniform es una perilla; el inspector y `set_shader_parameter` tocan la misma. Hoy la perilla maneja la pantalla completa.

La pregunta del día

"Modo retro", "todo gris en pausa",
"la pantalla late en rojo al recibir daño"...

¿Le pones un shader a CADA sprite, label y botón?

No. Le pones UN shader a una "ventana" que lee todo lo ya dibujado.

El shader de pantalla

El shader de pantalla: leer el frame y repintarlo



lee la foto... y la repinta a su manera — todo lo de la sesión 1 aplica, sobre el juego entero

viñeta · CRT · gris de pausa · viñeta de daño

solo pasa por el shader lo dibujado ANTES que él — por eso vive en un CanvasLayer encima

Viñeta y CRT (escena 12)

```
uniform sampler2D pantalla : hint_screen_texture, repeat_disable, filter_nearest;

void fragment() {
    vec3 color = texture(pantalla, SCREEN_UV).rgb;    // el frame, ya dibujado

    float dist = length(SCREEN_UV - vec2(0.5));      // ¡la viñeta es la escena 02!
    color = mix(color, vec3(0.0), smoothstep(0.35, 0.75, dist) * 0.8);

    COLOR = vec4(color, 1.0);
}
```

- `hint_screen_texture` → el uniform recibe la "foto" del frame.
- `SCREEN_UV` → mi posición **en la pantalla** (no en el rectángulo).

Perillas: teclas 1 (viñeta), 2 (CRT: fract + step sobre SCREEN_UV.y), 0 (nada) — el script cambia el uniform modo.

¿Qué le pasa a la pantalla? • 1

```
void fragment() {  
    vec3 c = texture(pantalla, SCREEN_UV).rgb;  
    COLOR = vec4(vec3(1.0) - c, 1.0);  
}
```

- A. El negativo: claro ↔ oscuro, los colores se invierten
- B. Blanco y negro
- C. Todo más brillante
- D. Pantalla en blanco

¿Qué da $1.0 - 0.0$? ¿Y $1.0 - 1.0$?

Pausa en gris (escena 13 • 🛠️ en vivo)

El **módulo 9** ya nos dio la pausa: `get_tree().paused` congela el árbol, y el overlay sobrevive por su `process_mode = Siempre`.

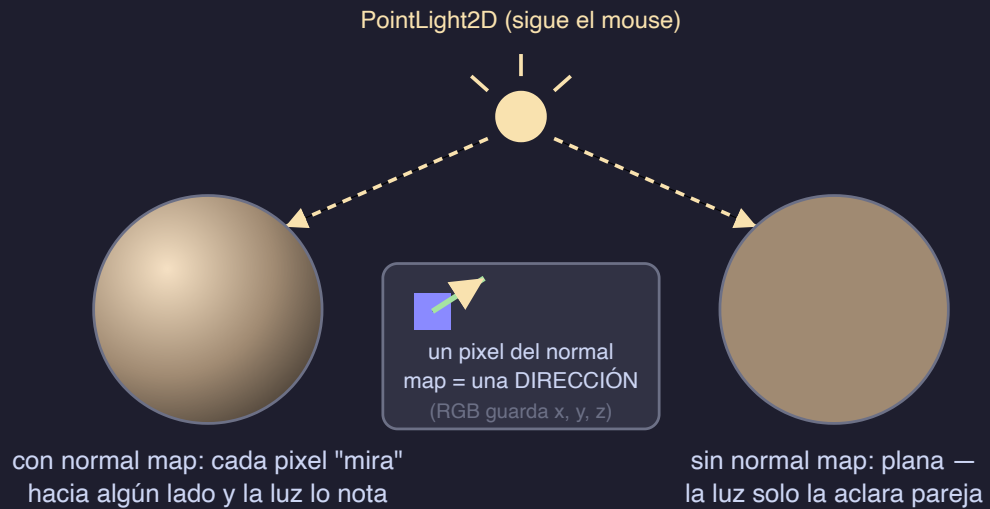
```
# pausa_gris.gd (completo) – Esc:  
get_tree().paused = pausado  
mat.set_shader_parameter("cantidad", 1.0 if pausado else 0.0)
```

```
// pausa_gris.gdshader – faltan DOS líneas (en vivo):  
// TODO: float lum = dot(color, vec3(0.299, 0.587, 0.114)); ← ¡la luminancia de la 07!  
// TODO: color = mix(color, vec3(lum), cantidad);
```

Estado roto visible: pausa sí, gris no. Dos líneas y el juego entero se desatura al pausar — la combinación módulo 9 + módulo 10.

Luz 2D + normal map (escena 14)

Normal map: la misma imagen, con volumen



Las piezas de la luz 2D

Sin escribir ningún shader — Godot trae el de la luz:

- **CanvasModulate** oscurece la escena (sin oscuridad no hay drama).
- **PointLight2D** ilumina (la nuestra sigue el mouse y "tiembla" como antorcha).
- **CanvasTexture** junta dos imágenes en una textura:
diffuse_texture (el color) + **normal_texture** (hacia dónde mira cada pixel).

```
Sprite2D.texture = CanvasTexture(diffuse = piedra_diffuse.png,  
                                normal  = piedra_normal.png)
```

En la escena 14: la MISMA imagen dos veces — con normal map reacciona al ángulo de la luz (volumen); sin él solo se aclara pareja (plana).

Quiz • 2

¿Qué guarda **cada pixel** de un normal map?

- A. Una dirección: hacia dónde "mira" la superficie en ese punto
- B. Una altura: qué tan elevado está ese punto
- C. El color de la sombra
- D. La transparencia

Por eso se ve azul-violeta: RGB codifica un vector (x, y, z) , y "mirar a la cámara" es $(0.5, 0.5, 1.0)$.

🎓 Ejercicio síntesis: la viñeta de daño (escena 15)

Pulsa H... nada. **Tres partes**, en dos archivos:

Parte	Archivo	Qué falta
A	<code>shaders/exercises/onda.gdshader</code>	declarar <code>uniform float intensidad</code>
B	el mismo	viñeta roja × <code>intensidad</code> (math de la 12)
C	<code>scripts/exercises/onda.gd</code>	empujar la perilla + tween (como la 10)

Es la sesión 2 + la sesión 3 en un solo efecto — y va directo a tu juego final. Reto extra: que lata con TIME mientras está activa.

Cierre del módulo: qué te llevas

El alcance creció una escala por sesión:

1. **Un rectángulo** — fragment, UV, mix/smoothstep, patrones, uniforms.
2. **Un sprite** — texture(), distorsión, vertex, y el puente con GDScript.
3. **La pantalla** — hint_screen_texture, pausa en gris, luz 2D.

Para el proyecto final, ya tienes hechos: hit flash, disolver de muerte, viñeta de daño, pausa en gris, contorno de selección, luz de antorcha. Son efectos de juego REAL — cópialos, ajústalos, hazlos tuyos.

La chuleta del módulo

Sesión 1	Sesión 2	Sesión 3
<code>COLOR</code> (vec4 RGBA)	<code>texture(TEXTURE, UV)</code>	<code>hint_screen_texture</code>
<code>UV</code> (0-1, Y abajo)	<code>TEXTURE_PIXEL_SIZE</code>	<code>SCREEN_UV</code>
<code>TIME</code>	<code>vertex()</code> / <code>VERTEX</code>	<code>CanvasModulate</code>
<code>mix</code> / <code>smoothstep</code> / <code>step</code>	<code>discard</code>	<code>PointLight2D</code>
<code>length</code> / <code>floor</code> / <code>mod</code>	<code>set_shader_parameter</code>	<code>CanvasTexture</code> (+normal)
<code>uniform</code> + hints	material compartido ⚠	<code>process_mode</code> (pausa)

Y los tres errores con punto flotante: `1.0`, no `1`. Siempre.